12 OPERATOR OVERLOADING • DETAILED SOLUTIONS

conceptual solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Conceptual Questions (Module 12: Operator Overloading)

Solution to Q1

- **Cannot be overloaded**: `::` (scope resolution), `.` (member access), `.*` (pointer-to-member access), `?:` (ternary conditional), `sizeof`, `typeid`, `alignof`, `noexcept`.
- **Must be non-static member functions**: `=`, `[]`, `()`, `→`.

Solution to Q2 (`c3 = c1 + c2` Translation)

- **As a member function**: Rewritten by the compiler as `c3 = c1.operator+(c2);` where `&c1` is passed as `this`.
- **As a `friend` function**: Rewritten by the compiler as `c3 = operator+(c1, c2);` where both `c1` and `c2` are explicit arguments.

Solution to Q3 (1 Parameter vs. 2 Parameters)

- A non-static member function always receives the invoking left-hand object implicitly via `this`. Adding a second explicit parameter to a binary member operator would give it 3 total operands (1 implicit + 2 explicit), violating the binary arity of `+`. A `friend` function is not a member function, so it has no `this` pointer and must receive both operands explicitly.

Solution to Q4 (Prefix `++obj` vs. Postfix `obj++`)

- Prefix `++` has signature `T& operator++()` (0 parameters), mutates `*this`, and returns `*this` by reference (avoiding an unnecessary copy).
- Postfix `++` has signature `T operator++(int)` (1 dummy `int` parameter), copies `*this` into a local variable `old`, increments `*this`, and returns `old` by value (since `old` is a local variable that will be destroyed on return, returning it by reference would create a dangling reference!).

Solution to Q5 (`operator<<` and `operator>>`)

- In `cout << obj`, the left-hand operand is `std::ostream` (`cout`), not `obj`. If `operator<<` were a member function of `MyClass`, you would have to write `obj << cout;`! To keep `cout` on the left, `operator<<` must be a non-member function taking `(ostream&, const MyClass&)`.

Solution to Q6 (Signed Overflow UB & `INT_MIN` Negation in `Show_Num()`)

- In C++23 (`[expr.pre]`), signed integer arithmetic overflow is Undefined Behavior.
- In 32-bit two's complement arithmetic, `INT_MIN` is `-2147483648` while `INT_MAX` is `+2147483647`. Because `+2147483648` is outside the range of `int`, evaluating `-imag` when `imag == INT_MIN`

overflows and causes Undefined Behavior.

- **Fix**: Cast `imag` to a wider signed integer (`long long`) before negating: `-(static_cast<long long>(imag))`.